PROTOFIRE

August 2025

Smart contract audit

Tokos - QuickSwap Adapter

Content

Project description

This project contains a suite of smart contract adapters designed to integrate the Aave V3 protocol with the QuickSwap DEX, which utilizes the CryptoAlgebra/Integral (Uniswap V3-style) architecture. These adapters function as intermediaries, enabling users to execute complex financial strategies that combine Aave's lending features with QuickSwap's swapping capabilities within single, atomic transactions. A core feature of these adapters is the use of Aave's flash loans to perform multi-step operations like debt swaps, liquidity migration, and collateralized repayments in a capital-efficient way. The contracts also support permit signatures to enhance user experience by reducing the number of required approval transactions.

Executive summary

Type	Utility contracts
Languages	Solidity
Methods	Architecture Review, Manual Review, Unit Testing, Functional Testing, Automated Review
Documentation	README.md
Repository	https://github.com/protofire/Somnia-AAVEV3-Fork

Reviews

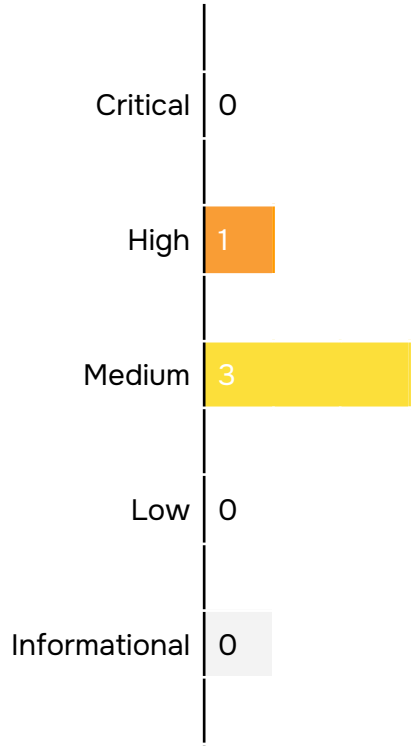
Date	Commit
05/09/2025	2364a771fc811cce8f74b5830111158cab873f5f
04/09/2025	4955bbcddc9bdc8ead96102df0e6d7cb7abe3403
28/08/2025	aff44cce46afc22b00cc9112663ab7b8be9a5d95

Scope

File Name
contracts/quickswap-adapters/BaseQuickSwapAdapter.sol
contracts/quickswap-adapters/BaseQuickSwapBuyAdapter.sol
contracts/quickswap-adapters/BaseQuickSwapSellAdapter.sol

contracts/quickswap-adapters/FlashLiquidationAdapterV3.sol
contracts/quickswap-adapters/QuickSwapDebtSwapAdapter.sol
contracts/quickswap-adapters/QuickSwapLiquiditySwapAdapter.sol
contracts/quickswap-adapters/QuickSwapRepayAdapter.sol
contracts/quickswap-adapters/QuickSwapWithdrawSwapAdapter.sol

Technical analysis and findings



Security findings

**** Critical

No critical severity issue found.

*** High

H01 - Excess Funds Lost During Repayment Without Swap

Impact	High
Likelihood	Medium

The swapAndRepay function in the QuickSwapRepayAdapter contract allows a user to repay a debt using their deposited collateral. The function contains a specific logic path for when the collateral asset is the same as the debt asset (collateralAsset == debtAsset), meaning no swap is required.

In this code path, the function correctly calculates the precise amount needed for repayment (amountToRepay). However, when it proceeds to withdraw funds from the user's aToken holdings via _pullATokenAndWithdraw, it incorrectly uses the collateralAmount parameter supplied by the user, instead of the calculated amountToRepay.

If a user provides a collateralAmount that is greater than their amountToRepay, the contract will withdraw the larger, incorrect amount. Since no swap is performed and there is no mechanism in this specific code path to return excess funds, the difference (collateralAmount - amountToRepay) will be stranded in the adapter contract, resulting in a direct and irrecoverable loss for the user.

This bug can cause a direct and irrecoverable loss of user funds.

Path: contracts/quickswap-adapters/QuickSwapRepayAdapter.sol: swapAndRepay()

Recommendation: In this case pass the amountToRepay value to the _pullATokenAndWithdraw function instead of the collateralAmount value.

Found in: aff44cc

Status: Fixed in 4955bbc

** Medium

M01 - Missing Token Approval Prevents Return of Leftover Collateral

Impact	Medium
Likelihood	High

The QuickSwapRepayAdapter contract contains two functions for handling repayments with collateral: the public swapAndRepay for direct calls, and the private _swapAndRepay for flash loan-based operations. Both functions are designed to return any leftover collateral to the user if the full amount is not consumed in the swap.

The public swapAndRepay function correctly implements this by first approving the Aave Pool to spend the leftover collateral, and then calling POOL.deposit(). However, the private _swapAndRepay function, which is executed in the context of a flash loan, omits the crucial approve call before attempting the deposit. Because the adapter has not granted the Pool an allowance, the POOL.deposit() call will fail, causing the entire flash loan transaction to revert.

Path: contracts/quickswap-adapters/QuickSwapRepayAdapter.sol: _swapAndRepay()

Recommendation: In this case make an approval to the Pool contract with collateralAmount - amountSold amount.

Found in: aff44cc

Status: Fixed in 4955bbc

M02 - Insufficient Allowance for Flash Loan Repayment with Extra Collateral

Impact	Medium
Likelihood	Low

In the QuickSwapDebtSwapAdapter contract, the executeOperation function handles the logic for repaying flash loans. When a debt swap requires extra collateral (the if (flashLoanParams.nestedFlashloanDebtAsset != address(0)) code path), the contract initiates a primary flash loan for the collateral asset.

The contract correctly calculates the total repayment amount for this primary flash loan, which is the principal (amounts[0]) plus the premium (premiums[0]). It successfully withdraws this total amount from the Aave pool. However, in the final step, when it should approve the Aave pool to reclaim the funds, it makes an incorrect call to _allowanceIfNeeded.

The function is called with only collateralAmount, omitting the premiums[0]. As a result, the Aave pool is not granted sufficient allowance to pull the full debt amount (collateralAmount + premiums[0]). This will cause the flash loan repayment to fail, reverting the entire transaction.

Path: contracts/quickswap-adapters/QuickSwapDebtSwapAdapter.sol: executeOperation()

Recommendation: In this case pass the collateralAmount + premiums[0] value to the _allowanceIfNeeded function instead of collateralAmount.

Found in: aff44cc

Status: Fixed in 4955bbc

M03 - Use of SafeERC20

The contracts directly call approve, transfer, and transferFrom on external ERC20 token contracts. These interactions do not utilize OpenZeppelin's SafeERC20 library, which is the industry best practice for handling external token calls.

Several widely-used ERC20 tokens, most notably Tether (USDT), do not strictly conform to the ERC20 standard; their approve and transfer functions are void (do not return a boolean). Calls from a Solidity contract expecting a return value will revert. Furthermore, the standard approve function is vulnerable to a known race condition attack.
The SafeERC20 library is specifically designed to solve both of these issues by providing wrappers (**safeApprove, safeTransfer, safeTransferFrom**) that handle non-standard tokens gracefully and mitigate the approve race condition. The absence of this library makes the contracts fragile and less secure.

Path: All contracts

Recommendation: Use SafeERC20 library from OpenZeppelin.

Found in: aff44cc

Status: Fixed in 2364a77

* Low

No low severity issue found.

Informational

No informational severity issue found.

Risk section

System Reliance on External Contracts: The entire functionality of the QuickSwap adapters is critically dependent on the correct and secure operation of external, unaudited (within this scope) contracts, specifically the Aave V3 Pool (POOL) and the QuickSwap V3 Router (QUICKSWAP_ROUTER). Any vulnerability, malicious action, administrative action (like pausing), or unexpected behavior in these core contracts will directly and severely impact the adapters. For instance:

- A bug or exploit in the QUICKSWAP_ROUTER could lead to failed swaps or a complete loss of funds sent for swapping.
- If the POOL contract is paused or compromised, all functions involving flash loans, deposits, withdrawals, or repayments will fail, potentially trapping user funds in an intermediate state if not handled carefully.

Flash Loan and Market Volatility Risks: The adapters for debt swaps, liquidity swaps, and repayments rely heavily on flash loans. This mechanism introduces specific risks related to market conditions:

- Denial of Service: A flash loan operation is atomic; if any step fails, the entire transaction reverts. A sudden spike in market volatility or low liquidity on QuickSwap could cause the internal swap to fail to acquire the necessary amount of tokens to repay the loan plus the premium. This would cause the user's transaction to revert, resulting in a loss of gas fees without achieving the desired outcome.
- Front-running/Sandwich Attacks: Since the swaps are executed in a predictable manner within a single transaction, they are susceptible to MEV (Maximal Extractable Value) attacks. A malicious actor could front-run the swap to manipulate the price unfavorably and then back-run it, extracting value at the user's expense.

Lack of User-Defined Slippage Control: The swap functions (_buyOnQuickSwap, _sellOnQuickSwap) do not allow the end-user to specify a slippage tolerance via a price limit. The limitSqrtPrice parameter in the swap calls is hardcoded to 0, effectively disabling a key price protection mechanism of the Uniswap V3-style protocol. This forces reliance on off-chain calculations for the minAmountOut/maxAmountToSwap parameters, which may not be sufficient to protect against all forms of price manipulation or high-volatility scenarios, potentially leading to users receiving a worse execution price than intended.

Approach and methodology

To establish a uniform evaluation, we define the following terminology in accordance with the OWASP Risk Rating
Methodology:

	**Likelihood** Indicates the probability of a specific vulnerability being discovered and exploited in real-world scenarios
	**Impact** Measures the technical loss and business repercussions resulting from a successful attack
	**Severity** Reflects the comprehensive magnitude of the risk, combining both the probability of occurrence (likelihood) and the extent of potential consequences (impact)

Likelihood and impact are divided into three levels: High H, Medium M, and Low L. The severity of a risk is a blend of these two factors, leading to its classification into one of four tiers: Critical, High, Medium, or Low.

When we identify an issue, our approach may include deploying contracts on our private testnet for validation through testing. Where necessary, we might also create a Proof of Concept PoC to demonstrate potential exploitability. In particular, we perform the audit according to the following procedure:

	**Advanced DeFi Scrutiny** We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs
	**Semantic Consistency Checks** We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
	**Security Analysis** The process begins with a comprehensive examination of the system to gain a deep understanding of its internal mechanisms, identifying any irregularities and potential weak spots.